

Random Forests

An Introduction with Regression and Classification Applications

Ryan McMurray

Melanie Jensen

Lucca

Math 3190: Fundamentals of Data Science

Rick Brown

Southern Utah University

April 21, 2026

Mathematical Foundation and Assumptions

Random forests are an ensemble learning method that construct a large collection of decision trees and aggregate their predictions to produce a single model. They are widely used for both regression and classification problems, particularly in settings where the relationship between predictors and the response is nonlinear or involves complex interactions. The central motivation for random forests is the reduction of variance: while individual decision trees tend to be highly variable, averaging many such trees—provided they are sufficiently decorrelated—yields a more stable and accurate predictor.

The construction of a random forest begins with decision trees, which partition the feature space into a set of disjoint regions and assign predictions based on those regions. In the regression setting, a tree partitions the predictor space into regions $R_1, \dots, R_J$ and defines a piecewise constant function of the form $\hat{f}(x) = \sum_{j=1}^J c_j \mathbf{1}(x \in R_j)$, where c_j is typically the mean of the observed responses within region R_j . These regions are chosen by recursively splitting the data to minimize the residual sum of squares. In the classification setting, predictions are instead made by majority vote within each region, and splits are chosen to reduce impurity, commonly measured by the Gini index or entropy. These criteria quantify how mixed the class labels are within a node, encouraging splits that produce more homogeneous subsets.

A random forest extends this framework by introducing randomness into the tree-building process. Specifically, each tree is trained on a bootstrap sample of the data, meaning that observations are sampled with replacement to form a new dataset of the same size. In addition, at each split within a tree, only a random subset of the available predictors is considered. This second source of randomness is critical, as it reduces the correlation between individual trees. Without it, trees built on bootstrap samples alone tend to be similar, particularly when there are strong predictors, limiting the effectiveness of averaging.

Predictions from a random forest are obtained by aggregating the outputs of the individual trees. For regression, this is done by averaging, $\hat{f}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$, while for classification, the predicted class is determined by majority vote across the trees. The theoretical benefit of this approach can be understood through the variance of the ensemble. If each tree has variance σ^2 and pairwise correlation ρ , then the variance of the averaged predictor is approximately $\text{Var}(\hat{f}) \approx \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. This expression highlights that increasing the number of trees reduces variance, but the reduction is most effective when the trees are weakly correlated. The random feature selection mechanism plays a key role in achieving this decorrelation.

Although random forests are often described as making few assumptions, they do rely on several important conditions. The training observations are assumed to be independent and identically distributed, and the model implicitly assumes that the distribution of the training data matches that of future observations. Furthermore, the predictors must contain sufficient information about the response for meaningful patterns to be learned. Like other tree-based methods, random forests are also limited in their ability to extrapolate beyond the range of the observed data, as predictions are constructed from averages of observed outcomes.

An additional practical advantage of random forests is the availability of out-of-bag error estimation. Because each tree is trained on a bootstrap sample, approximately one-third of the observations are omitted from the training set for that tree. These unused observations can be used to evaluate predictive performance by averaging predictions over the subset of trees in which the observation was not included. This provides an efficient and nearly unbiased estimate of generalization error without the need for a separate validation set.

Despite their strong empirical performance, random forests are not without limitations. Their ensemble nature makes them less interpretable than simpler models, and commonly used measures of feature importance can be difficult to interpret or biased in the presence of correlated predictors.

Additionally, training a large number of trees can be computationally demanding, particularly for large datasets or high-dimensional problems. Random forests may also struggle when many predictors are irrelevant, and, as noted earlier, they do not extrapolate well beyond the range of the training data.

In summary, random forests build upon the strengths of decision trees while addressing their primary weakness—high variance—through averaging and randomization. By combining bootstrap sampling with random feature selection, they produce a flexible and robust modeling approach that performs well across a wide range of regression and classification problems.

Regression Example

To demonstrate how random forest regression behaves in practice, this section walks through the process of building, tuning, and evaluating a random forest regression model on the Ames Housing dataset. This dataset contains 2,930 observations on residential home sales in Ames, Iowa, along with 80 predictor variables covering physical characteristics, quality ratings, location, and sale conditions. The response variable of interest is `Sale_Price`, the final sale price of each home in dollars. The dataset is well suited to a random forest demonstration because it contains a mix of continuous and categorical predictors, several groups of correlated variables, and nonlinear relationships that would be difficult to capture with a linear model without substantial feature engineering.

Setup and Data Preparation

The analysis uses the `randomForest` package in R for model fitting and tuning, along with `tidyverse` packages for data manipulation and visualization. The `AmesHousing` package provides a cleaned version of the data through its `make_ames()` function, which handles factor level

encoding and removes the need for most preprocessing.

```
library(tidyverse)
library(randomForest)
library(AmesHousing)
library(kableExtra)

ames <- make_ames()

set.seed(2026)
split <- sample(nrow(ames), size = 0.8 * nrow(ames))
train <- ames[split, ]
test <- ames[-split, ]
```

An 80/20 train test split was used, with 80 percent of the data reserved for model fitting and the remaining 20 percent held out for evaluation. A fixed random seed was set before the split to ensure reproducibility, which is especially important for random forests given that both the bootstrap sampling process and the random feature selection at each split introduce stochasticity into the model.

Fitting the Initial Model

The initial random forest model was fit using all 80 predictors to demonstrate one of the practical strengths of the method, which is its ability to handle high-dimensional data without requiring manual variable selection or transformation. The model was built with the default 500 trees and `mtry` set to the regression default of $p/3$, where p is the number of predictors. This default corresponds to 26 variables being randomly considered at each split in each tree.

```
set.seed(2026)
rf_model <- randomForest(
  Sale_Price ~ .,
  data = train,
  ntree = 500,
  mtry = floor((ncol(train) - 1) / 3),
  importance = TRUE
)
```

The `importance = TRUE` argument instructs the function to compute permutation-based variable importance during training, which will be used in a later section to examine which predictors the model is relying on most. Computing importance during the initial fit avoids the need to refit the model later solely for that purpose.

The output of the fitted model reports that 500 trees were built with 26 variables considered at each split, an out-of-bag mean squared error of approximately 6.26×10^8 , and a percent of variance explained of roughly 89 percent. These out-of-bag statistics come from a built-in validation mechanism specific to bagged methods. Because each tree is trained on a bootstrap sample that omits roughly one third of the training observations, those omitted observations can be used to evaluate the prediction quality of each tree without requiring a separate validation set. Aggregating these predictions across all trees in which an observation was out-of-bag produces a nearly unbiased estimate of generalization error, which is reported as the out-of-bag MSE.

Determining the Appropriate Number of Trees

Although 500 trees is the default, the choice of `ntree` is worth examining because it has implications for both model stability and computational cost. Unlike most hyperparameters, `ntree` is not a tuning parameter in the traditional sense. Adding more trees cannot cause a random forest to overfit, because each additional tree simply refines the average prediction rather than increasing model complexity. The question is therefore not which value of `ntree` minimizes error, but rather how many trees are needed for the out-of-bag error to stabilize.

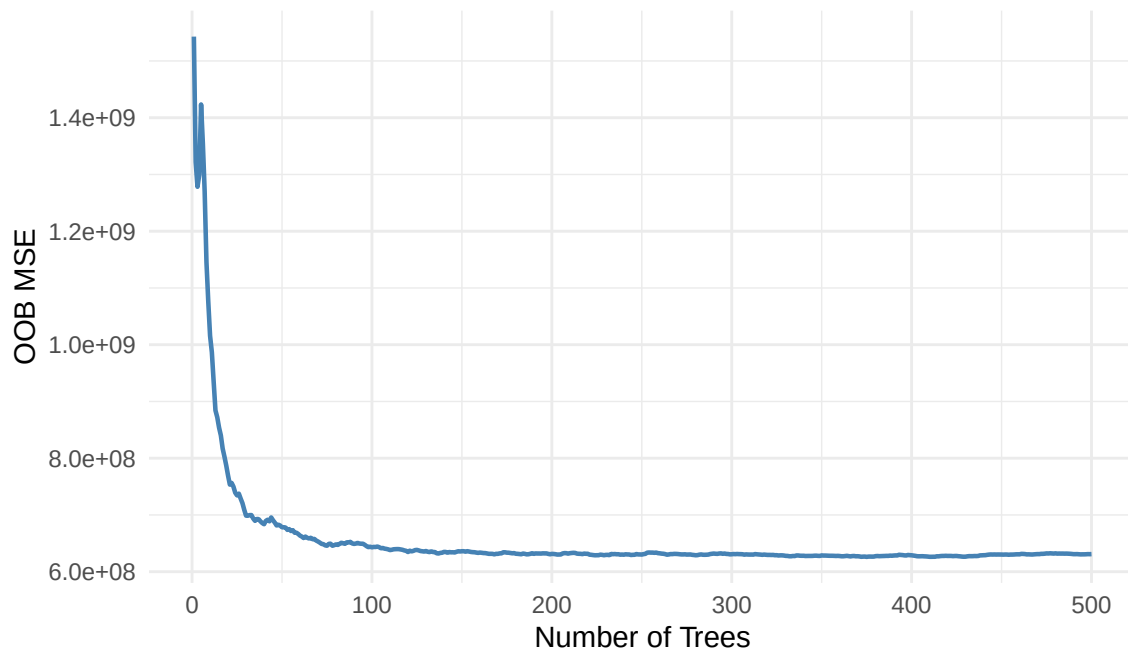


Figure 1: Out-of-bag mean squared error as a function of the number of trees in the forest.

Figure 1 shows how the out-of-bag MSE evolves as trees are added to the forest. The error drops rapidly over the first 100 trees and stabilizes near 200 trees, after which additional trees produce no meaningful reduction in error. This pattern is consistent with the theoretical behavior of bagged ensembles, where variance is reduced by averaging but the benefit diminishes as more trees are added.

Tuning the Number of Variables Per Split

The more meaningful hyperparameter in a random forest is `mtry`, which controls how many predictors are randomly considered as candidates at each split. Lower values produce more decorrelated trees at the cost of individual tree accuracy, while higher values allow each tree to consider stronger splits but make the trees more similar to one another. The `tuneRF` function in the `randomForest` package searches across `mtry` values by adjusting the starting value up and down by a chosen step factor and selecting the value that minimizes out-of-bag error.

```

set.seed(2026)
tuneRF(
  x = train |> select(-Sale_Price),
  y = train$Sale_Price,
  ntreeTry = 200,
  stepFactor = 1.5,
  improve = 0.01,
  trace = FALSE,
  plot = TRUE
)

```

The tuning procedure tested `mtry` values of 18, 26, and 39. The default value of 26 produced the lowest out-of-bag MSE, with the alternatives performing less than one percent worse. This result illustrates a well-known property of random forests, which is their insensitivity to hyperparameter choices within a reasonable range. The default $p/3$ heuristic was confirmed to be near-optimal for this dataset, and no additional tuning was warranted.

A final model was fit using the tuned specification. Because the convergence analysis in the previous section showed that out-of-bag error stabilized well before 500 trees, the final model was built with only 200 trees to reduce computational cost while preserving essentially identical predictive performance.

```

set.seed(2026)
tuned_rf_model <- randomForest(
  Sale_Price ~ .,
  data = train,
  ntree = 200,
  mtry = 26,
  importance = TRUE
)

```

Model Evaluation

Evaluation of a random forest regression model differs meaningfully from evaluation of a linear regression model. Because random forests make essentially no distributional assumptions about the data or the errors, traditional diagnostic procedures such as checking for normality of residuals

or constant variance do not apply in the same way. Instead, evaluation focuses on direct measures of predictive performance on held-out data and on visual inspection of where the model performs well or poorly.

Table 1: Performance metrics for the tuned random forest on held-out test data.

Metric	Value
Test RMSE	\$25,154
Test R^2	0.911
OOB RMSE	\$25,127

The test RMSE of approximately \$25,154 indicates that on average the model predictions are off by around that amount on houses with a mean sale price of roughly \$180,000, which corresponds to about a 14 percent relative error. The test R^2 of 0.911 indicates that the model explains 91 percent of the variance in sale price on houses it was never trained on. Perhaps most notably, the out-of-bag RMSE of \$25,127 differs from the test RMSE by only \$27, which confirms that the out-of-bag error provides a reliable estimate of generalization performance. This close agreement is a practical validation of the theoretical claim that out-of-bag estimates are nearly unbiased approximations of test error, and it suggests that in settings where holding out a test set is undesirable, the out-of-bag estimate alone would have produced essentially the same conclusion about model quality. This close alignment also serves as a robustness check on the method itself, confirming that the training procedure produced a model that generalizes consistently rather than one that happened to fit the training data well by chance. For random forest regression, this kind of internal validation is a key diagnostic tool, particularly when working with smaller datasets where setting aside a test set would meaningfully reduce the information available for model fitting.

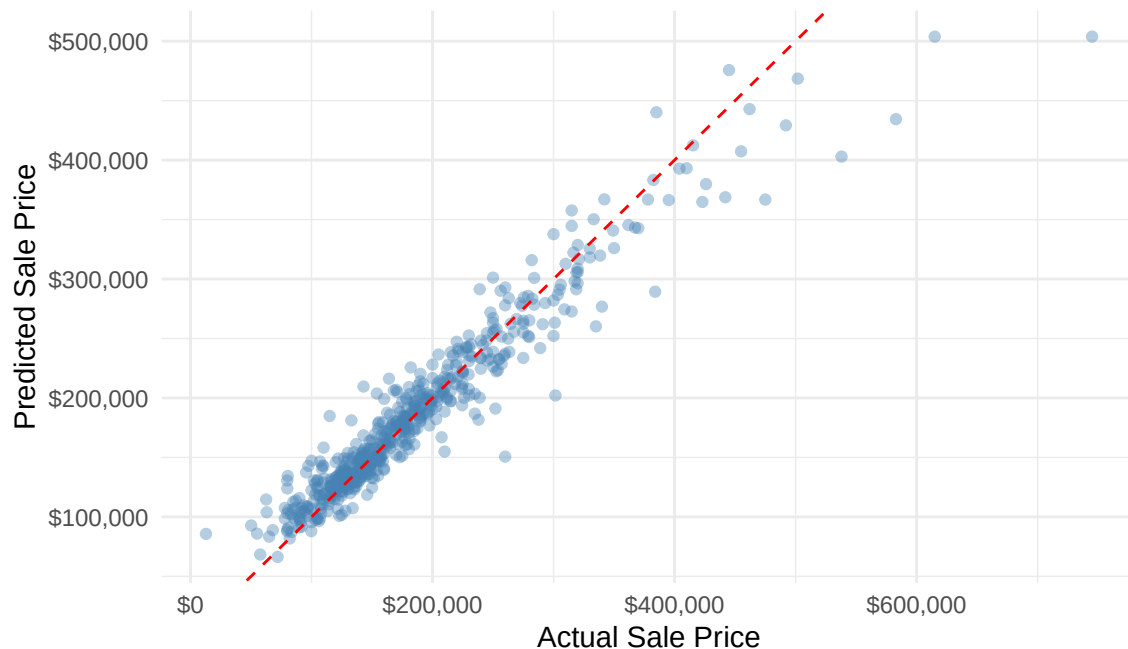


Figure 2: Predicted versus actual sale prices on the held-out test set. The dashed red line represents perfect prediction.

Figure 2 plots predicted sale prices against actual sale prices on the test set. In a well-performing model, points should cluster tightly around the diagonal line with no systematic drift above or below it, which would indicate that predictions are accurate across the entire range of the response. Points cluster tightly around the diagonal through most of the price range, with the densest region of data falling between \$100,000 and \$300,000 where the model performs best. Two patterns become visible at the upper end of the price range. The spread of the points widens, indicating that prediction uncertainty grows for more expensive homes, and the points shift systematically below the diagonal, indicating that the model tends to underpredict the most expensive homes. Both patterns share a common cause. Random forest predictions are constructed by averaging responses in the terminal nodes of each tree, which means no prediction can exceed the largest value observed in the training data. For homes at or beyond the upper range of the training set, the model is both less informed by data and structurally incapable of extrapolating upward. This is a known limitation of tree-

based methods and is important to acknowledge when applying random forests to problems where extreme values matter.

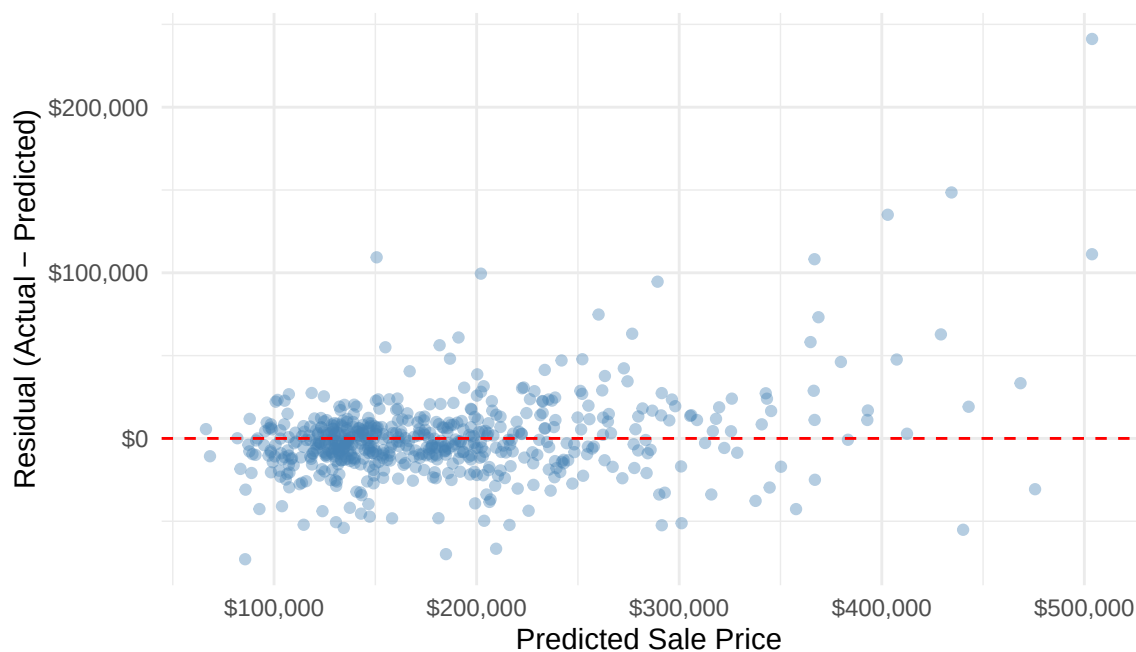


Figure 3: Residuals from the tuned random forest model, defined as actual minus predicted sale price, plotted against predicted sale price.

The residual plot in Figure 3 makes the same patterns explicit. Residuals fan out as predictions increase, and at the highest predicted values the residuals are predominantly positive, which is the mirror image of the underprediction visible in Figure 2. In a linear regression context, this pattern would be interpreted as a violation of the homoscedasticity assumption and would prompt remedial action such as a log transformation of the response. In the random forest context, where no such assumption is being made, the pattern is instead interpreted descriptively as evidence that the model performs less reliably at the extremes of the price distribution.

Variable Importance

One of the practical advantages of random forests over linear models is the availability of built-in variable importance measures that describe which predictors the model is relying on most. This

analysis uses permutation-based importance, reported as percent increase in mean squared error when a variable is permuted. The procedure takes the out-of-bag observations and randomly shuffles the values of a single predictor while holding all other predictors fixed, then measures how much the out-of-bag MSE increases as a result. A variable whose permutation causes a large increase in MSE is one the model is relying on heavily for accurate predictions, while a variable whose permutation changes MSE very little is one the model is effectively ignoring.

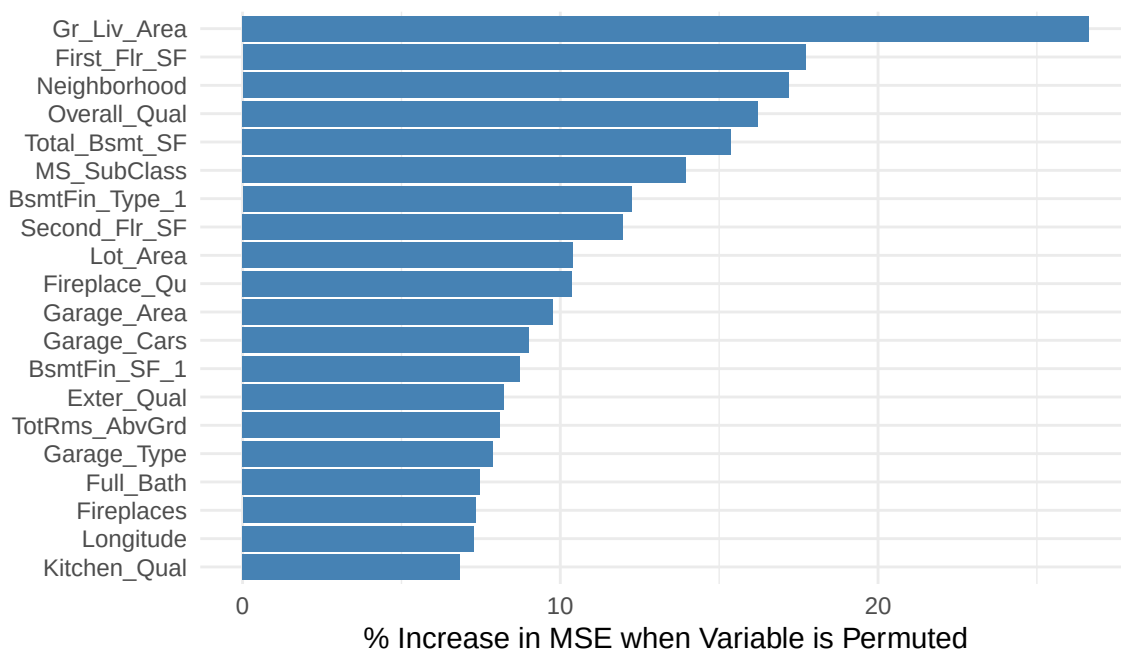


Figure 4: Top 20 predictors ranked by permutation-based variable importance, measured as the percent increase in out-of-bag MSE when the variable is permuted.

Figure 4 shows the top 20 predictors ranked by permutation importance. The most important variables are **Gr_Liv_Area** (above-ground living area), followed by **First_Flr_SF**, **Neighborhood**, **Total_Bsmt_SF**, and several other size and location variables. This ranking aligns with real-world intuition about housing prices, where square footage and location are typically the strongest drivers of value. The alignment between the model's most important variables and what a domain expert would prioritize serves as a useful sanity check that the model is learning meaningful patterns rather

than spurious ones.

Below the top 20 predictors, importance scores drop toward zero, and a number of variables show slightly negative values. A negative percent increase in MSE does not mean that the variable actively harms predictions. Instead, it reflects the fact that the permutation procedure introduces its own random variation, and when a variable contributes essentially no predictive signal, the measured importance fluctuates around zero and can land on either side. In practical terms, variables at the bottom of the ranking are either carrying information already captured by correlated predictors or have no meaningful relationship with sale price, and the model is not relying on them regardless.

This finding has a useful implication for model simplification. Because many of the 80 predictors in the model contribute little to prediction, a reduced model fit on only the top 15 to 20 predictors would likely perform nearly as well as the full model. In settings where interpretability, training speed, or data collection cost matter, using variable importance to guide feature selection is a common and defensible practice. For the purposes of this demonstration, the full model is retained to illustrate how random forests handle high-dimensional data without requiring that selection step.

Summary

The random forest regression model built on the Ames Housing dataset achieves strong predictive performance on held-out test data, explaining approximately 91 percent of the variance in sale price with a test RMSE of about \$25,000. The close agreement between out-of-bag and test error confirms the theoretical property that out-of-bag estimation provides a reliable approximation of generalization error. Tuning analysis showed that the default hyperparameters were near-optimal, consistent with the well-known robustness of random forests to parameter choices. Variable importance analysis produced a ranking that aligns with domain intuition, with size and location

predictors dominating the model. The primary limitation observed was the model's tendency to underpredict the highest-priced homes, which reflects the structural inability of tree-based methods to extrapolate beyond the training data. Overall, the example demonstrates that random forests are well suited to problems with many mixed-type predictors, nonlinear relationships, and correlated features, where linear models would require substantial preprocessing to perform comparably. The method is most appropriate in applied settings where predictive accuracy is the primary concern, such as price prediction, risk scoring, ecological modeling, and medical diagnostics. It is particularly valuable when working with datasets that contain a large number of candidate predictors whose individual relationships with the response are not well understood, because the algorithm can identify informative variables without requiring the analyst to specify those relationships in advance. The method is less appropriate when the goal is formal statistical inference or when predictions must extend beyond the range of the observed data, in which case a parametric model with explicit assumptions about the relationship between predictors and the response would be more suitable.

Classification Example